# TILE PLACEMENT SYSTEM

## OVERVIEW

The Tile Placement System handles when the player places tiles (I think it would kind of be strange if a system called the “tile placement system” did something else).

The primary scripts involved in placing tiles are BuildingSystem.cs and ObjectDrag.cs. The Grid System (which has its own doc) is also involved in object placement. The BuildingSystem class sits on the Grid game object. There is an instance of the ObjectDrag class on each and every tile.

## HOW TILE PLACEMENT WORKS

Ignoring a few complexities, this is the general way that the tile placement system works: Whenever the player clicks on a tile’s button, the tile select panel passes the prefab tile of the clicked button to the building system. The building system then instantiates that tile. When instantiated, the tile is in drag mode; in this mode, the ObjectDrag class on the tile makes the tile follow the mouse. It determines where the mouse is by getting the mouse world position from the building system. The next time the player left clicks, the building system checks to see the tile can be placed in its current position (using the CanBePlaced() function). If it can, then it tells the Object Drag class to place the tile, using the Place() function. When that function runs, the tile exits dragging mode, stops following the mouse, and adds itself to the grid using GridManager.GM.AddObject().

In order to allow the player to place multiple tiles without having to repeatedly click that tile’s button, the building system automatically creates another instance of that tile’s prefab after the previous tile is placed. That new tile is in drag mode and will begin following the mouse.

# HOW DRAG PLACEMENT WORKS

A slightly different system than usual tile placement is used to allow the player to click and drag to place tiles. The object drag script always keeps track of when its tile moves from one spot on the grid to another, and it also always tells the building system when that happens. The building system usually tunes out the ObjectDrag class's blabbering about its movements, except when the player is holding down the left mouse button. If the player is holding down the mouse when the tile moves, the building system places the active tile. This is what enables click and dragging. *Every time the mouse moves to a new tile while the left mouse button is held down, the active tile is placed*.

# DETERMINING IF A TILE CAN BE PLACED

The Building System uses this internal function to determine if an active tile (the unplaced tile following the mouse) can be placed:

***public bool CanBePlaced(bool attemptingToPlaceTile)***

It returns true if the active tile can be placed, and it returns false otherwise. If the input boolean "attemptingToPlaceTile" is true, then it will display errors to the user about why the tile can't be placed. There are a series of checks it takes to determine a tile's placeability. First, it checks if the mouse is over the screen using isMouseOverScreen(), which ensures that the mouse isn't off the edge of the game window and that it isn't over any UI elements. Then it checks to make sure the active object isn't currently over the void using isObjectOverVoid(), which performs a raycast straight down over the top of the object to see if it's currently above the ground or not. After that, it makes sure that the current chunk had been purchased by the player, using ChunkPurchaseManager.current.ActiveChunkIsPurchased. Next, it checks to make sure that the player isn't trying to place the tile over another tile of the same type using isActiveObjectOverlappingSameTileType(). Then the building system checks that there's not too much carbon to place the tile, there's enough money to place the tile, and there are enough employees to place the tile all in one step, using activeTile.CheckIfTileIsPlaceable(), which is a function on the Tile.cs script.

Finally, the building system's CanBePlaced function uses the ObjectDrag class's function CanBePlacedOnOverlappingTile() to determine if this tile can either overlap with or is able to destroy the tiles it's currently sitting on top of. (More information on determining when objects can destroy / overlap with other objects in the next section.)

The CanBePlacedOnOverlappingTile() function only determines if the tile can be placed on top of other tiles, not whether or not it will destroy the other tiles there; that will be handled in the next section, "Determining When Tiles Can Overlap With & Destroy Other Tiles".

At the end of all of those checks, if everything comes back fine, the CanBePlaced function returns true and the the building system places the tile, calling the ObjectDrag class's function, Place().

# DETERMINING WHEN TILES CAN OVERLAP WITH & DESTROY OTHER TILES

There are two main questions to answer here: 1) Are two tiles allowed to overlap? 2) If a tile is being placed on a tile it's not allowed to overlap with, can it destroy that tile and take its place?

The ObjectDrag script is in charge of answering both these questions. It answers the first one with its function AllowObjectOverlap(GameObject otherTile). It answers the second one with its function CanDestroyOverlappingTile(GameObject otherTile). In both cases, the function returns a true or false boolean, and "otherTile" is the tile that ObjectDrag is overlapping with.

The rules for which tiles are allowed to overlap are simple, because there's only one kind of permitted overlap: Everything can overlap with grass tiles, except for road tiles.

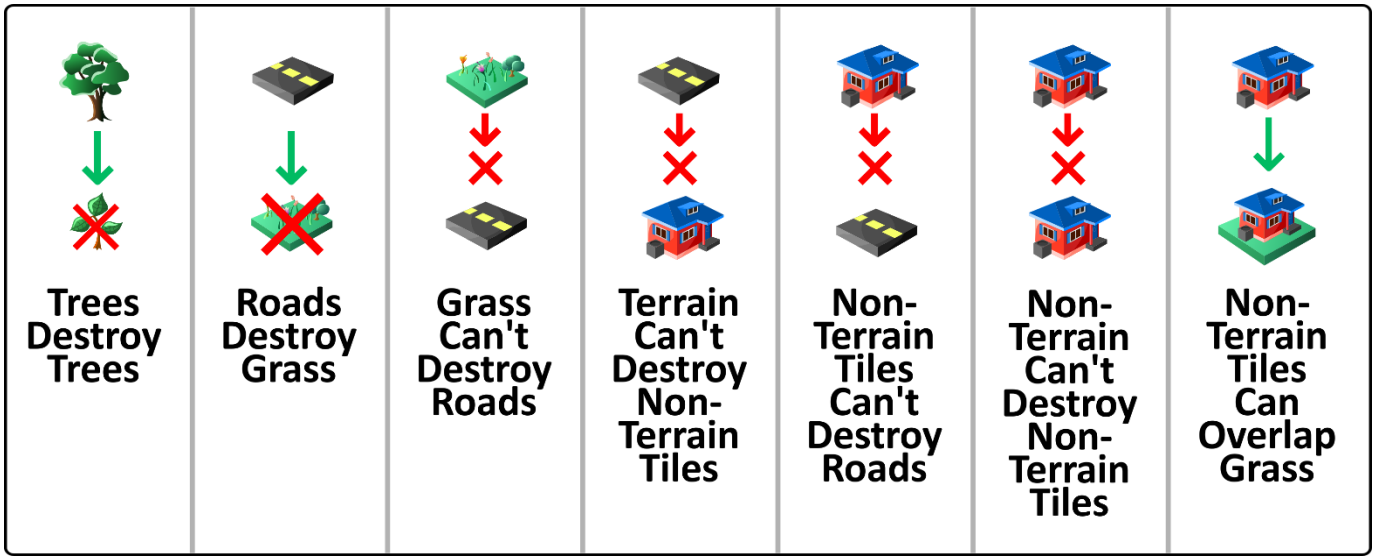
There are also only a few rules for when a tile can destroy another tile. The first is that different kinds of trees can destroy each other. The second is that roads can destroy grass. There are no other scenarios in which tiles are allowed to destroy other tiles.

The CanBePlaced function in the Building System, which determines if a tile can be placed in a certain spot, comes back true if the unplaced tile can either overlap with or destroy the tiles at its grid location. But just because CanBePlaced allows a tile to be placed, that doesn't mean that the tile it's overlapping with can stay. For example, a road tile can be placed on top of a grass tile, but the grass tile must still be destroyed.

The destruction of overlapping tiles is handled by the ObjectDrag class, in its Place() function. So, once a tile is placed by the building system, and the ObjectDrag class's Place() function is called, it will again use the function AllowObjectOverlap(GameObject otherTile). If either the overlapping terrain or overlapping non-terrain tile isn't allowed to overlap, it will destroy that tile.

# TILE OVERLAP / DESTRUCTION DIAGRAM

This diagram has some examples of how the overlap and tile destruction rules work in action:



# INTERNAL API

### BuildingSystem.cs

Here are some important functions of the BuildingSystem class, whose singleton is accessible at BuildingSystem.current.

#### bool CanBePlaced(bool attemptingToPlaceTile)

Returns true if the active tile can be placed in its current location; returns false otherwise. Runs a series of checks to determine placeability. If the input boolean "attemptingToPlaceTile" is true, then it will display placement errors to the user should a tile placement fail.

#### public void deselectActiveObject()

Deselects the active tile and destroys its unplaced prefab. Called when the player right clicks or when the tile select panel deselects the active tile.

#### public void activeObjectMovedToNewTile()

Places a tile if the player is holding down the left mouse button. Called from ObjectDrag to alert the building system that the active tile has moved to a new grid location.

#### public static Vector3 GetMouseWorldPosition()

Returns the world coordinates of the mouse. It gets this by sending out a raycast from the camera to the ground, looking to collide the game object "Raycast Hitter," which is an invisible plane that lies across the entire ground.

#### public static bool isMouseOverScreen()

Returns true if the mouse is over the world. Returns false if it's over any of the UI panels or if it's off the edge of the game window, gallivanting around, doing who knows what.

#### public Vector3 SnapCoordinateToGrid(Vector3 position)

Snaps the input "position" to the grid and returns the snapped coordinates.

#### public void InitializeWithObject(GameObject prefab)

Begins the tile placement process with the input "prefab." If for example, the apartment is being placed, the input prefab will be the apartment's prefab. This function is called from the Tile Select Panel.

#### public static bool isObjectOverVoid()

Returns true if the active object isn't over the ground. Checks if it's over the ground by raycasting straight down over the active tile, looking to collide with the "ground" object.

#### public bool isActiveObjectOverlappingSameTileType()

Returns true if the active, unplaced tile is currently overlapping a tile of the same type.

#### public bool ShouldActiveTileMaterialBeValid()

Determines whether the material of the unplaced tile should be red or not.

### ObjectDrag.cs

These are some of the important public functions of the ObjectDrag class.

#### public void Place()

Called from building system -- places the tile that ObjectDrag is attached to. It forces the tile to stop following the mouse, sets its placed state, sets its placed material, and deletes tiles that aren't allowed to overlap with it.

#### public void updateTileMaterialValidity()

Sets the correct material for the tile, depending on what state it's in.

#### public bool AllowObjectOverlap(GameObject otherTile)

Returns true if the tile that ObjectDrag is attached to can overlap with the input "otherTile." This means that the two tiles can coexist on the same grid square, like a grass tile and a house tile.

#### public bool CanDestroyOverlappingTile(GameObject otherTile)

Returns true if the tile ObjectDrag is attached to a tile that can destroy "otherTile". Basically, trees can destroy each other, and anything can destroy grass.

#### public bool CanBePlacedOnOverlappingTile()

Returns true if the tile can be placed on top of the tiles it's currently overlapping with. That means it can either overlap with or destroy the terrain it's on top of, and it can either overlap with or destroy the non-terrain tile it's on top of.

---

-Graydon Bruyn, May 2026